

WSL(windows subsystem for linux)

WSL 2 완벽 가이드: Windows에서 Linux를 만나다

1.1 WSL(Windows Subsystem for Linux) 이란?

WSL 개요 & 역사

WSL1 vs WSL2 비교

왜 WSL을 사용할까? (개발자 관점에서의 이점)

1.2 WSL 2 장점

전체적인 성능 개선

향상된 호환성

Windows와의 시너지

1.3 WSL 2 주요 기능

파일 시스템 통합

GUI 애플리케이션 실행 (WSLg)

다중 리눅스 배포판 운영

시스템 자원 제어

1.4 설치 방법

WSL 활성화

배포판 설치 및 업데이트

WSL1 ↔ WSL2 전환

Linux 시스템 설정 및 커스터마이징

1.5 한국어 환경 설정 및 기본 애플리케이션 설정

Gedit 설정

한국어 설정

D-Bus 활성화

한글 입력기 및 폰트 설치

Nautilus 설정

Bridge 네트워크 설정

Mirror 네트워크 설정

1.6 Docker와 함께 사용하기

WSL2 기반 Docker 구조

설치 및 설정

리눅스 터미널에서 Docker 명령어 사용

1.7 GPU 리소스 활용하기

WSL2 GPU 지원 개요

CUDA on WSL2 설정

Docker + NVIDIA Toolkit

테스트 실습

1.8 USB 장치 연결하기 (아두이노, ATmega 등)

핵심 원리: USB/IP와 usbipd-win

1단계: Windows에 usbipd-win 설치

2단계: WSL 2(Linux)에 USB/IP 클라이언트 도구 설치

3단계: USB 장치 연결 및 공유

4단계: WSL 2에서 장치 확인 및 사용

USB 장치 연결 해제

1.9 VS Code 연동으로 개발 환경 완성하기

왜 VS Code와 WSL을 함께 사용해야 하는가?

1단계: WSL 확장 프로그램 설치

2단계: WSL에 연결하기

3단계: 개발 워크플로우 체험하기

WSL 2 완벽 가이드: Windows에서 Linux를 만나다

1.1 WSL(Windows Subsystem for Linux)이란?

WSL 개요 & 역사

WSL(Windows Subsystem for Linux)은 Windows 운영체제에서 별도의 가상 머신(Virtual Machine)이나 듀얼 부팅 없이 Linux 배포판(예: Ubuntu, Debian)을 직접 실행할 수 있도록 지원하는 Windows의 기능입니다. 개발자들은 Windows 환경을 떠나지 않고도 Linux의 강력한 커맨드 라인 도구, 유틸리티, 애플리케이션을 원활하게 사용할 수 있습니다.

역사:

- **WSL 1 (2016년 출시):** Microsoft가 Linux의 시스템 호출(System Call)을 Windows NT 커널의 시스템 호출로 실시간 번역해주는 '번역 계층(Translation Layer)'을 개발하여 구현했습니다. 이는 Linux 커널을 직접 실행하는 것이 아니었기 때문에 완벽한 호환성을 보장하지 못했고, 특히 파일 시스템 I/O 성능에 한계가 있었습니다.
- **WSL 2 (2019년 출시):** 이러한 한계를 극복하기 위해 Microsoft는 접근 방식을 완전히 바꾸었습니다. WSL 2는 경량화된 가상 머신(VM) 기술인 Hyper-V를 기반으로 실제 Linux 커널을 직접 실행합니다. 이로 인해 완벽한 시스템 호출 호환성과 비약적인 성능 향상, 특히 파일 시스템 성능 개선을 이루어냈습니다.

WSL1 vs WSL2 비교

구분	WSL 1	WSL 2
아키텍처	번역 계층 (Translation Layer)	경량 가상 머신 (Hyper-V 기반)
Linux 커널	자체 번역 (리눅스 커널 없음)	실제 리눅스 커널
성능 (파일 I/O)	상대적으로 느림 (NTFS와 변환 과정 필요)	최대 20배 빠름 (ext4 파일 시스템 직접 사용)
시스템 호출 호환성	일부 호환되지 않는 syscall 존재	완벽한 호환성
네트워킹	Windows와 동일한 IP 주소 공유	별도의 가상 네트워크 인터페이스 (별도 IP)
Windows와의 통합	파일 시스템 접근 속도가 빠름	파일 시스템 접근 속도가 상대적으로 느림 (/mnt/*)
초기 부팅 속도	매우 빠름	약간의 초기 부팅 시간 필요

핵심 차이: WSL 1이 '번역'을 통해 Linux 환경을 흉내 냈다면, WSL 2는 '가상화'를 통해 실제 Linux 커널을 구동합니다. 이로 인해 호환성과 성능 면에서 WSL 2가 압도적인 우위를 가지게 되었습니다. 현재는 특별한 이유가 없는 한 WSL 2 사용이 강력히 권장됩니다.

왜 WSL을 사용할까? (개발자 관점에서의 이점)

- **통합 개발 환경:** Windows의 GUI 환경(VS Code, JetBrains IDE 등)과 Linux의 강력한 CLI(Command-Line Interface) 환경을 동시에 사용할 수 있습니다. Windows에서 코드를 작성하고, Linux 터미널에서 바로 컴파일, 실행, 배포하는 워크플로우가 가능해집니다.
- **속도와 가벼움:** Docker Desktop이나 VMware, VirtualBox와 같은 전통적인 가상 머신에 비해 WSL 2는 훨씬 가볍고 빠르게 시작됩니다. 시스템 자원을 덜 소모하면서도 네이티브에 가까운 성능을 제공합니다.
- **생산성 향상:** 웹 개발, 클라우드, 컨테이너 기술(Docker, Kubernetes) 등 현대 개발 생태계는 대부분 Linux를 기반으로 합니다. WSL을 사용하면 복잡한 설정 없이 이러한 기술들을 Windows에서 손쉽게 도입하고 학습할 수 있습니다.
- **비용 절감:** 별도의 Linux 서버나 PC를 구비할 필요 없이, 사용하던 Windows PC 한 대로 두 운영체제의 장점을 모두 누릴 수 있습니다.

1.2 WSL 2 장점

전체적인 성능 개선

WSL 2의 가장 큰 장점은 단연 성능입니다. Hyper-V 기술을 기반으로 최적화된 경량 유틸리티 VM 위에서 실제 Linux 커널을 실행하기 때문에 전반적인 컴퓨팅 성능이 크게 향상되었습니다. 특히 파일 시스템 속도는 혁신적으로 개선되었습니다. WSL 1에서는 Linux 파일

시스템 연산이 Windows NT 커널을 거쳐야 했지만, WSL 2에서는 Linux 네이티브 파일 시스템인 ext4 위에서 직접 동작합니다. 그 결과, `git clone`, `npm install`, `apt update` 등 파일 I/O가 많은 작업에서 최대 20배까지 성능이 향상되었습니다.

향상된 호환성

WSL 2는 실제 Linux 커널을 사용하므로 시스템 호출(System Call) 호환성이 100% 보장됩니다. 이는 WSL 1에서 번역 계층의 한계로 실행할 수 없었던 Docker, FUSE(Filesystem in Userspace) 등과 같은 애플리케이션들을 완벽하게 지원한다는 의미입니다. 이제 개발자들은 "Windows에서 안 되겠지?"라는 고민 없이 대부분의 Linux용 소프트웨어를 자신 있게 사용할 수 있습니다.

Windows와의 시너지

WSL 2는 가상화 기술을 사용함에도 불구하고 Windows와의 통합성을 잃지 않았습니다.

- **파일 시스템 공유:** Windows 탐색기에서 `\\wsl$` 주소를 입력하면 설치된 Linux 배포판의 전체 파일 시스템에 접근할 수 있습니다. 반대로 Linux 터미널에서는 `/mnt/c`, `/mnt/d` 등의 경로를 통해 Windows의 드라이브에 쉽게 접근할 수 있습니다.
- **GUI 애플리케이션 실행:** WSLg(WSL GUI) 기능이 내장되어, 별도의 X 서버 설정 없이도 Linux용 GUI 애플리케이션(Gedit, GIMP, IDE 등)을 설치하고 실행하면 바로 Windows 화면에 창이 나타납니다. 이는 마치 네이티브 Windows 앱처럼 자연스럽게 동작합니다.

1.3 WSL 2 주요 기능

파일 시스템 통합

WSL 2는 Windows와 Linux 간의 원활한 파일 접근을 지원하여 통합된 작업 환경을 제공합니다.

- **Windows에서 Linux 파일 접근:**
 - Windows 탐색기를 열고 주소창에 `\\wsl$` 또는 `\\wsl.localhost` 를 입력합니다.
 - 설치된 배포판 이름(예: Ubuntu-22.04)의 폴더가 나타나며, 이를 클릭하면 Linux의 루트(`/`) 파일 시스템으로 바로 접근할 수 있습니다.
 - VS Code와 같은 편집기에서도 이 경로를 통해 Linux 내 파일을 직접 열고 수정할 수 있습니다.
- **Linux에서 Windows 파일 접근:**

- Linux 터미널에서 `/mnt/` 디렉토리로 이동하면 Windows의 드라이브들이 마운트되어 있습니다.
 - C 드라이브: `/mnt/c`
 - D 드라이브: `/mnt/d`
- **성능 팁:** 최상의 파일 I/O 성능을 위해서는 프로젝트 파일을 Linux 파일 시스템(예: `~/projects`) 내에 위치시키는 것이 좋습니다. `/mnt/c` 경로의 파일을 다루는 것은 운영 체제 간 파일 시스템 변환으로 인해 속도가 저하될 수 있습니다.

GUI 애플리케이션 실행 (WSLg)

WSL 2에는 WSLg(Windows Subsystem for Linux GUI)라는 기능이 기본적으로 포함되어 있습니다. 이를 통해 사용자는 복잡한 설정 없이 Linux GUI 앱을 실행할 수 있습니다.

```
# 예시: Ubuntu에 Gedit 텍스트 편집기 설치 및 실행
sudo apt update
sudo apt install gedit -y

# 설치 후 터미널에 'gedit' 입력
gedit
```

위 명령어를 실행하면, 잠시 후 Windows 데스크톱에 Gedit 창이 나타납니다. 시작 메뉴에도 자동으로 등록되어 일반 Windows 프로그램처럼 검색하고 실행할 수 있습니다.

다중 리눅스 배포판 운영

WSL 2는 여러 개의 Linux 배포판을 동시에 설치하고 관리하는 기능을 지원합니다.

- **배포판 설치:**
 - Microsoft Store에서 Ubuntu, Debian, Kali Linux 등을 검색하여 설치하거나, 터미널에서 명령어로 설치할 수 있습니다.

```
# 설치 가능한 배포판 목록 확인
wsl --list --online

# 특정 배포판 설치 (예: Debian)
wsl --install -d Debian
```

- **설치된 배포판 확인 및 관리:**

```
# 설치된 배포판 목록과 상태(버전, 실행 여부) 확인
wsl --list --verbose
```

- **임포트/익스포트:** 자신만의 설정을 마친 배포판을 tar 파일로 내보내 백업하거나 다른 PC로 옮겨 가져올 수 있습니다.

```
# Ubuntu-22.04 배포판을 D:\backup\my-ubuntu.tar로 내보내기
wsl --export Ubuntu-22.04 D:\backup\my-ubuntu.tar

# D:\backup\my-ubuntu.tar 파일을 MyNewUbuntu라는 이름으로 가져
오기
wsl --import MyNewUbuntu D:\wsl\MyNewUbuntu D:\backup\my-ub
untu.tar
```

시스템 자원 제어

WSL 2는 경량 VM으로 동작하므로 시스템 자원을 관리하는 명령어를 제공합니다.

- **WSL 머신 시작/종료/재부팅:**

```
# 실행 중인 모든 배포판과 WSL 2 VM 종료
wsl --shutdown

# 특정 배포판 종료 (예: Ubuntu-22.04)
wsl --terminate Ubuntu-22.04
```

종료된 배포판은 터미널을 다시 실행하면 자동으로 시작됩니다.

- **.wslconfig를 통한 고급 설정:**
 - Windows의 사용자 폴더 (`C:\Users\<YourUser>`)에 `.wslconfig` 파일을 생성하여 WSL 2 VM의 자원을 세밀하게 제어할 수 있습니다.

```
# 예시: .wslconfig 파일 내용
[wsl2]
memory=4GB    # WSL 2 VM에 할당할 최대 메모리
processors=2  # 할당할 프로세서 수
swap=0        # 스왑 파일 비활성화
localhostForwarding=true
```

파일 수정 후에는 `wsl --shutdown` 명령으로 VM을 재시작해야 설정이 적용됩니다.

1.4 설치 방법

WSL 활성화

WSL을 설치하기 전에 Windows의 관련 기능을 먼저 활성화해야 합니다.

1. **관리자 권한으로 PowerShell 또는 Windows 터미널 실행:** 시작 메뉴에서 PowerShell을 검색하여 마우스 오른쪽 버튼을 클릭하고 '관리자 권한으로 실행'을 선택합니다.
2. **WSL 및 가상 머신 플랫폼 기능 활성화:** 아래 명령어를 입력하고 실행합니다.

```
dism.exe /online /enable-feature /featurename:Microsoft-Windows-Subsystem-Linux /all /norestart  
dism.exe /online /enable-feature /featurename:VirtualMachinePlatform /all /norestart
```

1. **컴퓨터 재부팅:** 위 기능들을 완전히 활성화하려면 컴퓨터를 다시 시작해야 합니다.

배포판 설치 및 업데이트

가장 간단한 설치 방법은 `wsl --install` 명령어를 사용하는 것입니다. 이 명령어는 필요한 모든 단계를 자동으로 처리해 줍니다.

- **관리자 권한 터미널에서 설치 명령어 실행:**

```
wsl --install
```

이 명령어는 기본적으로 Ubuntu 배포판을 설치합니다. 다른 배포판을 설치하고 싶다면 `-d` 옵션을 사용합니다. (예: `wsl --install -d Debian`)

- **Linux 시스템 설정 (최초 실행 시):**
 - 설치가 완료되고 Ubuntu(또는 선택한 배포판) 창이 나타나면, Linux 시스템에서 사용할 사용자 이름(Username)과 비밀번호(Password)를 설정하라는 메시지가 표시됩니다.
 - **중요:** 여기서 설정하는 계정은 Windows 계정과 별개입니다. `sudo` 명령어를 사용할 때 이 비밀번호가 필요하므로 잘 기억해야 합니다.
- **패키지 업데이트:** 사용자 설정이 끝나면, 가장 먼저 시스템 패키지를 최신 상태로 업데이트하는 것이 좋습니다.

```
sudo apt update && sudo apt upgrade -y
```

WSL1 ↔ WSL2 전환

이미 WSL 1을 사용 중이거나, 특정 배포판의 버전을 변경하고 싶을 때 사용하는 명령어입니다.

- 설치된 배포판과 버전 확인:

```
wsl --list --verbose
# NAME STATE VERSION
#* Ubuntu-22.04 Running 2
```

- 버전 변경:

```
# Ubuntu-22.04 배포판을 버전 2로 설정
wsl --set-version Ubuntu-22.04 2
```

```
# 반대로 버전 1로 설정
wsl --set-version Ubuntu-22.04 1
```

- 향후 설치될 배포판의 기본 버전을 WSL 2로 설정:

```
wsl --set-default-version 2
```

Linux 시스템 설정 및 커스터마이징

개발 생산성을 높이기 위해 터미널 환경을 꾸밀 수 있습니다. `oh-my-zsh` 는 가장 인기 있는 터미널 커스터마이징 프레임워크 중 하나입니다.

- Zsh 설치:

```
sudo apt install zsh -y
```

- Oh My Zsh 설치:

```
sh -c "$(curl -fsSL https://raw.githubusercontent.com/ohmyzsh/ohmyzsh/master/tools/install.sh)"
```

- **테마 및 플러그인 설정:**

- 설치가 완료되면 `~/.zshrc` 파일이 생성됩니다. 이 파일을 편집하여 테마를 변경하거나 유용한 플러그인(예: `git`, `zsh-autosuggestions`, `zsh-syntax-highlighting`)을 추가할 수 있습니다.
- **예시 (테마 변경):** `ZSH_THEME="agnoster"`

1.5 한국어 환경 설정 및 기본 애플리케이션 설정

WSL 2 환경에서 한국어 입력과 GUI 애플리케이션을 원활히 사용하려면 추가적인 설정이 필요합니다. 아래는 `nautilus`, `gedit`, 그리고 한국어 입력기 설정 방법을 포함합니다.

Gedit 설정

`gedit` 는 Linux에서 널리 사용되는 텍스트 편집기로, WSLg를 통해 Windows 데스크톱에서 실행할 수 있습니다.

```
sudo apt install gedit
```

- **북마크 파일 생성:** `gedit` 에서 파일 탐색기 북마크를 사용하려면 아래 명령어로 필요한 파일을 생성합니다.

```
touch ~/.gtk-bookmarks
sudo touch /root/.gtk-bookmarks
```

한국어 설정

WSL 2에서 한국어 입력과 로케일 설정을 위해 아래 단계를 따릅니다. 기본적으로 D-Bus가 비활성화되어 있어 이를 활성화하고, `ibus` 입력기를 설정해야 합니다.

D-Bus 활성화

D-Bus는 애플리케이션 간 통신을 위한 메시지 버스 시스템으로, 한국어 입력기(`ibus`)가 제대로 동작하려면 활성화해야 합니다.

```
# D-Bus 실행
eval $(dbus-launch --sh-syntax)
```

- **자동 실행 설정:** D-Bus가 매번 터미널 실행 시 자동으로 시작되도록 `~/.bashrc` 에 아래 내용을 추가합니다.

```
# D-Bus 세션 버스 주소 설정
if [ -z "$DBUS_SESSION_BUS_ADDRESS" ]; then
    eval $(dbus-launch --sh-syntax)
fi
```

한글 입력기 및 폰트 설치

`ibus-hangul` 과 필요한 폰트를 설치하여 한국어 입력을 활성화합니다.

```
sudo apt install -y ibus ibus-hangul dconf-cli dbus-x11
sudo apt install fonts-nanum fonts-noto-cjk
sudo apt install -y language-pack-ko
sudo update-locale LANG=ko_KR.UTF-8
source /etc/default/locale
```

- **ibus 데몬 설정:** `ibus` 입력기가 자동으로 시작되도록 `~/.bashrc` 에 아래 내용을 추가합니다.

```
# ibus 데몬 시작
if [ -z "$(pgrep ibus-daemon)" ]; then
    ibus-daemon -drx
fi

export GTK_IM_MODULE=ibus
export QT_IM_MODULE=ibus
export XMODIFIERS="@im=ibus"
```

- **ibus 설정:**
 - `ibus-setup` 명령어를 실행하여 입력기 설정 창을 엽니다.
 - Input Method 란에서 "한국어 - Hanguk"을 추가합니다.
 - Preferences에서 단축키(예: `Ctrl+Space` 또는 `한/영` 키)를 설정합니다.

```
ibus-setup
```

Nautilus 설정

`nautilus` 는 Linux의 대표적인 파일 관리자입니다. WSLg를 통해 Windows 환경에서 실행 가능합니다.

```
sudo apt install nautilus
```

설치 후 터미널에서 `nautilus` 를 입력하면 파일 탐색기가 Windows 데스크톱에 나타납니다.

Bridge 네트워크 설정

WSL 2의 기본 네트워킹은 NAT(Network Address Translation) 방식이지만, 외부 네트워크와 직접 통신하려면 브리지(bridge) 모드를 설정할 수 있습니다.

- **Hyper-V 외부 스위치 추가:**

- Windows에서 Hyper-V 관리자 또는 PowerShell을 통해 외부 스위치(예: `bridge`)를 생성합니다.
- **참고:** 노트북의 Wi-Fi 네트워크 카드에서는 브리지 모드가 제대로 작동하지 않을 수 있습니다. 이 경우 랜선(LAN) 연결이나 USB Wi-Fi dongle을 사용하세요.

- **.wslconfig 설정:**

- Windows의 사용자 폴더(`C:\Users\<YourUser>`)에서 `.wslconfig` 파일을 편집하여 네트워크 설정을 추가합니다.

```
# PowerShell에서 .wslconfig 열기
notepad "$env:USERPROFILE\.wslconfig"
```

- `.wslconfig` 에 아래 내용을 추가합니다.

```
[wsl2]
networkingMode=bridged
vmSwitch=bridge # 생성한 스위치 이름
```

- 설정 적용을 위해 WSL을 재시작합니다.

```
wsl --shutdown
```

Mirror 네트워크 설정

Mirror 네트워크 모드는 Windows 11 22H2 이상에서 지원되는 최신 네트워킹 방식으로, WSL 2 인스턴스가 Windows 호스트와 동일한 네트워크 인터페이스를 공유하게 합니다. 이를 통해 localhost 주소를 더 자연스럽게 공유하고, IPv6 지원이 향상되며, 외부에서 WSL로 직접 접근이 가능해집니다.

Mirror 모드의 장점

- **동일한 IP 주소 공유:** WSL 2와 Windows가 동일한 네트워크 인터페이스와 IP 주소를 공유하여 네트워크 설정이 단순해집니다.
- **향상된 localhost 접근:** WSL에서 실행 중인 서버에 Windows의 localhost로 바로 접근 가능하며, 그 반대도 마찬가지입니다.
- **IPv6 지원:** 완전한 IPv6 지원으로 최신 네트워킹 환경에 적합합니다.
- **외부 접근 가능:** 같은 네트워크의 다른 장치에서 WSL 내부 서비스에 직접 접근할 수 있습니다

Mirror 모드 설정 방법

- **Windows 버전 확인:** Mirror 모드를 사용하려면 Windows 11 버전 22H2 이상이 필요합니다. PowerShell에서 `winver` 명령어로 확인할 수 있습니다.
- **.wslconfig 파일 편집:** Windows 사용자 폴더(`C:\Users\<YourUser>`)에서 `.wslconfig` 파일을 생성하거나 편집합니다.

```
# PowerShell에서 .wslconfig 열기
notepad "$env:USERPROFILE\.wslconfig"
```

- **Mirror 네트워크 설정 추가:** `.wslconfig` 파일에 아래 내용을 추가합니다.

```
[wsl2]
networkingMode=mirrored
dnsTunneling=true
firewall=true
autoProxy=true
```

- **설정 옵션 설명:**
 - `networkingMode=mirrored` : Mirror 모드 활성화
 - `dnsTunneling=true` : DNS 쿼리를 WSL에서 Windows로 터널링하여 DNS 해석 문제 방지

- `firewall=true` : Windows 방화벽 규칙을 WSL에도 적용
- `autoProxy=true` : Windows의 프록시 설정을 WSL에 자동으로 적용
- **WSL 재시작**: 설정을 적용하려면 WSL을 완전히 종료한 후 다시 시작합니다.

```
wsl --shutdown
```

설정 확인

WSL 터미널에서 IP 주소를 확인하여 Windows와 동일한 네트워크 인터페이스를 사용하는 지 확인합니다.

```
# WSL에서 IP 주소 확인
ip addr show eth0

# Windows에서 IP 주소 확인 (PowerShell)
ipconfig
```

두 환경에서 표시되는 IP 주소가 동일하다면 Mirror 모드가 정상적으로 작동하는 것입니다.

주의사항



ros2 사용시 **ros2 daemon start** 를 하지 않으면 토픽이 보이지 않을 수 있습니다.

- **방화벽 설정**: Mirror 모드에서는 Windows 방화벽 규칙이 WSL에도 적용되므로, WSL 서비스에 외부 접근이 필요한 경우 방화벽 규칙을 적절히 설정해야 합니다.
- **포트 충돌**: Windows와 WSL이 동일한 네트워크 스택을 공유하므로, 같은 포트를 동시에 사용할 수 없습니다.
- **VPN 호환성**: 일부 VPN 소프트웨어는 Mirror 모드와 호환성 문제가 있을 수 있으므로, VPN 사용 시 네트워크 연결을 확인해야 합니다.
- Vscode의 codex 나 gemini code assist 계정 설정시 인증 오류가 생길 수 있습니다. nat로 인증하고 돌아와야 합니다.

1.6 Docker와 함께 사용하기

WSL2 기반 Docker 구조

Docker Desktop for Windows는 WSL 2를 백엔드로 활용하여 Linux 컨테이너를 실행하는 방식을 채택했습니다. 이는 Docker에 날개를 달아준 혁신적인 변화였습니다.

- **구조:** Docker Desktop은 WSL 2 내에 최적화된 `docker-desktop` 및 `docker-desktop-data` 배포판을 설치합니다. Docker 엔진, CLI, Kubernetes 등 모든 Docker 구성 요소가 이 WSL 2 환경 안에서 직접 실행됩니다.
- **성능 개선:** 실제 Linux 커널 위에서 Docker 데몬이 동작하므로, Hyper-V를 직접 사용하던 레거시 방식보다 훨씬 빠르고 안정적입니다. 특히 볼륨 마운트 시 파일 시스템 성능이 크게 향상되어 애플리케이션 빌드 및 실행 속도가 빨라집니다.

설치 및 설정

1. **Docker Desktop 다운로드 및 설치:** Docker 공식 홈페이지에서 Docker Desktop for Windows를 다운로드하여 설치합니다.
2. **WSL 2 기반 엔진 활성화:**
 - 설치 과정 또는 설치 후 Docker Desktop의 Settings > General 메뉴로 이동합니다.
 - "Use the WSL 2 based engine" 옵션이 반드시 체크되어 있는지 확인합니다. (최신 버전에서는 기본값입니다.)
3. **특정 배포판과 통합:**
 - Settings > Resources > WSL Integration 메뉴로 이동합니다.
 - Docker 명령어를 사용하고자 하는 설치된 배포판(예: Ubuntu-22.04)을 활성화(Enable)합니다.

이 설정을 통해 해당 배포판의 터미널에서 Windows의 Docker CLI를 그대로 사용할 수 있게 됩니다.

리눅스 터미널에서 Docker 명령어 사용

WSL Integration 설정이 완료되면, Ubuntu 터미널을 열고 바로 `docker` 명령어를 사용할 수 있습니다.

- **동작 확인:**

```
# Docker 버전 확인
docker --version
# Docker: Docker Engine - Community 20.10.17, ...
```

```
# Docker 정보 확인
docker info
```

- **컨테이너 실행 (Hello World):**

```
docker run hello-world
```

- **웹 서버 컨테이너 실행 (Nginx):**

```
# Nginx 컨테이너를 백그라운드에서 실행하고, 로컬 포트 8080과 컨테이너 포트 80을 연결
docker run --name my-nginx -d -p 8080:80 nginx
```

이제 Windows의 웹 브라우저에서 <http://localhost:8080> 으로 접속하면 Nginx 시작 페이지가 나타나는 것을 확인할 수 있습니다. WSL 2의 네트워킹 기능 덕분에 이 모든 것이 매끄럽게 동작합니다.

1.7 GPU 리소스 활용하기

WSL2 GPU 지원 개요

WSL 2의 가장 강력한 기능 중 하나는 NVIDIA CUDA를 통한 GPU 가속을 지원한다는 것입니다. 이를 통해 Windows에서 직접 머신러닝/딥러닝 모델 학습, 데이터 분석, 고성능 컴퓨팅(HPC) 작업을 수행할 수 있습니다. WSL 2는 Windows에서 NVIDIA GPU의 하드웨어 가속 기능을 Linux 환경에 직접 제공하는 기술(GPU Paravirtualization, GPU-PV)을 적용한 유일한 환경입니다. 이는 별도의 가상 머신에서 GPU를 사용하기 위해 복잡한 'PCI-e Passthrough' 설정을 할 필요가 없다는 것을 의미합니다.

CUDA on WSL2 설정

GPU를 WSL 2에서 사용하려면 몇 가지 사전 설정이 필요합니다.

1. **NVIDIA 드라이버 설치 (WSL 2 지원 버전):**

- NVIDIA CUDA on WSL 드라이버 다운로드 페이지로 이동하여 자신의 GPU 모델에 맞는 최신 드라이버를 다운로드하고 설치합니다. 일반 Game Ready 드라이버가 아닌, WSL을 명시적으로 지원하는 드라이버여야 합니다.

2. **WSL 2 및 Linux 배포판 설치:** 앞서 설명한 방법대로 WSL 2를 설치하고 원하는 배포판(주로 Ubuntu)을 설치합니다.

3. **WSL 커널 업데이트 (필요 시):**

```
wsl --update
```

이 명령어를 통해 WSL 2의 Linux 커널을 최신 버전으로 업데이트하여 GPU 지원 관련 최신 기능이 포함되도록 합니다.

Docker + NVIDIA Toolkit

Docker 컨테이너 안에서 GPU를 사용하려면 `nvidia-container-toolkit` 이 필요합니다.

- **NVIDIA Container Toolkit 설치 (Ubuntu 터미널 내에서):**

```
curl -fsSL https://nvidia.github.io/libnvidia-container/gpg
key | sudo gpg --dearmor -o /usr/share/keyrings/nvidia-cont
ainer-toolkit-keyring.gpg \
    && curl -s -L https://nvidia.github.io/libnvidia-containe
r/stable/deb/nvidia-container-toolkit.list | \
    sed 's#deb https://#deb [signed-by=/usr/share/keyrings/
nvidia-container-toolkit-keyring.gpg] https://#g' | \
    sudo tee /etc/apt/sources.list.d/nvidia-container-toolk
it.list
sudo apt-get update
sudo apt-get install -y nvidia-container-toolkit
```

- **Docker 데몬 재시작:** Toolkit 설정을 적용하기 위해 Docker를 재시작해야 합니다. 가장 간단한 방법은 Docker Desktop을 종료했다가 다시 실행하는 것입니다.

테스트 실습

- **nvidia-smi로 GPU 인식 확인 (WSL 2 터미널):**

```
nvidia-smi
```

위와 같이 Windows에서 보던 것과 동일한 GPU 정보(드라이버 버전, CUDA 버전, GPU 모델 등)가 출력된다면 성공적으로 인식된 것입니다.

- **CUDA 벤치마크 컨테이너 실행:**

```
docker run --rm --gpus all nvidia/cuda:11.6.2-base-ubuntu2
0.04 nvidia-smi
```

- `-gpus all` 옵션은 이 컨테이너가 호스트(Windows)의 모든 GPU를 사용하도록 지정하는 옵션입니다. 이 명령어가 오류 없이 `nvidia-smi` 결과를 출력한다면, 이제 당신은 WSL 2와 Docker 컨테이너 환경에서 GPU 가속을 마음껏 활용할 준비가 된 것입니다.

1.8 USB 장치 연결하기 (아두이노, ATmega 등)

기본적으로 WSL 2는 가상화 환경이므로 Windows에 연결된 USB 장치에 직접 접근할 수 없습니다. 하지만 USB/IP 기술을 활용하면 네트워크를 통해 USB 장치를 공유하는 것처럼 Windows 호스트의 USB 장치를 WSL 2 게스트에 연결할 수 있습니다. 이를 위해 `usbipd-win` 프로젝트를 사용합니다.

핵심 원리: USB/IP와 usbipd-win

- **USB/IP:** USB 장치를 IP 네트워크를 통해 공유하기 위한 프로토콜입니다. 로컬에 연결된 USB 장치를 원격지의 컴퓨터에서 사용하는 것처럼 만들어줍니다.
- **usbipd-win:** Windows에서 USB/IP 서버 역할을 수행하게 해주는 오픈소스 프로젝트입니다. 이를 통해 Windows에 연결된 USB 장치를 WSL 2의 Linux 환경으로 넘겨줄 수 있습니다.

1단계: Windows에 usbipd-win 설치

- 관리자 권한으로 PowerShell 또는 Windows 터미널을 실행합니다.
- `winget` 패키지 관리자를 사용하여 `usbipd-win` 을 설치합니다.

```
winget install --interactive --exact dorssel.usbipd-win
```

설치 과정에서 권한 상승 요청이 나타날 수 있습니다.

2단계: WSL 2(Linux)에 USB/IP 클라이언트 도구 설치

- Ubuntu (또는 Debian 계열) 터미널을 엽니다.
- USB/IP 클라이언트 도구와 하드웨어 데이터베이스를 설치합니다.

```
sudo apt update
sudo apt install linux-tools-virtual hwdata
```

3단계: USB 장치 연결 및 공유

- **연결할 USB 장치 확인 (PowerShell):**

- 아두이노 우노나 ATmega128 보드를 PC의 USB 포트에 연결합니다.
- 관리자 권한 PowerShell에서 다음 명령어를 실행하여 현재 연결된 USB 장치 목록과 BUS ID를 확인합니다.

```
usbipd list
```

목록에서 연결하려는 장치(예: Arduino Uno, USB-SERIAL CH340 등)를 찾아 BUSID를 기록해 둡니다.

- **USB 장치 공유 (PowerShell):**

```
# 예시: BUSID가 4-2인 장치를 공유
usbipd bind --busid 4-2
```

- `-force` 옵션을 추가하면 다른 드라이버가 사용 중이어도 강제로 바인딩할 수 있습니다.
- **USB 장치 WSL에 연결 (PowerShell):**

```
usbipd attach --wsl --busid 4-2
```

4단계: WSL 2에서 장치 확인 및 사용

- **WSL 2 터미널에서 장치 확인:**
 - `lsusb` 명령어를 실행하여 USB 장치가 정상적으로 인식되었는지 확인합니다.

```
lsusb
```

Windows에서 보았던 장치 정보가 목록에 나타나야 합니다.

- 아두이노와 같은 시리얼 통신 장치는 `/dev/` 디렉토리에 `ttyUSB0` 또는 `ttyACM0` 과 같은 이름으로 나타납니다. 다음 명령어로 확인할 수 있습니다.

```
ls -l /dev/tty*
```

- **Arduino CLI 또는 PlatformIO 사용:**
 - 이제 WSL 2 환경은 USB 장치를 네이티브하게 인식하므로, 리눅스용 개발 도구를 그대로 사용할 수 있습니다.
- **Arduino CLI 설치 예시:**

```
# Arduino CLI 설치
curl -fsSL https://raw.githubusercontent.com/arduino/arduino-cli/master/install.sh | sh
sudo mv bin/arduino-cli /usr/local/bin/

# 보드 목록 확인 (연결된 포트가 보여야 함)
arduino-cli board list
```

이제 `arduino-cli compile --fqbn <보드정보> <스케치경로>` 및 `arduino-cli upload -p /dev/ttyACM0 --fqbn <보드정보> <스케치경로>` 와 같은 명령어로 컴파일과 업로드를 모두 WSL 2 터미널에서 직접 수행할 수 있습니다.

USB 장치 연결 해제

작업이 끝난 후에는 장치 연결을 해제하여 Windows에서 다시 사용할 수 있도록 해야 합니다.

- PowerShell에서 다음 명령어를 실행합니다.

```
# 예시: BUSID가 4-2인 장치 연결 해제
usbipd detach --busid 4-2
```

이 과정을 통해 WSL 2 환경에서도 임베디드 개발, 펌웨어 프로그래밍 등 다양한 하드웨어 관련 작업을 Windows를 떠나지 않고 수행할 수 있습니다.

1.9 VS Code 연동으로 개발 환경 완성하기

WSL 2의 진정한 잠재력은 Visual Studio Code(VS Code)와 만났을 때 발휘됩니다. VS Code의 WSL 확장 프로그램을 사용하면 Windows에서 VS Code의 편리한 GUI를 사용하면서, 코드 편집, 실행, 디버깅 등 모든 작업을 WSL 2의 Linux 환경에서 수행할 수 있습니다. 이는 Windows와 Linux의 장점만을 결합한 최고의 개발 환경을 제공합니다.

왜 VS Code와 WSL을 함께 사용해야 하는가?

- **완벽한 통합:** VS Code UI는 Windows에서 실행되지만, 터미널, 디버거, IntelliSense(자동 완성) 등은 모두 WSL의 Linux 환경에서 동작하여 마치 Linux에서 직접 개발하는 듯한 경험을 제공합니다.
- **최고의 성능:** 프로젝트 파일들을 WSL 2의 Linux 파일 시스템(ext4)에 저장하고 작업하므로, Git, npm/yarn, 컴파일러 등이 네이티브 속도로 동작합니다.

- **원활한 워크플로우:** Windows에서 VS Code로 코드를 수정하고, 내장된 WSL 터미널에서 바로 Linux 명령어를 실행하여 테스트하고 배포하는 과정이 끊김 없이 이어집니다.

1단계: WSL 확장 프로그램 설치

1. Windows에 Visual Studio Code를 설치합니다.
2. VS Code를 실행하고 왼쪽의 확장 프로그램(Extensions) 탭을 엽니다 (단축키: `Ctrl+Shift+X`).
3. 검색창에 `WSL` 을 입력하고, Microsoft에서 제공하는 WSL 확장 프로그램을 찾아 설치합니다.

2단계: WSL에 연결하기

WSL 확장 프로그램을 설치하면 여러 가지 방법으로 WSL 환경에 접속할 수 있습니다.

- **방법 1: WSL 터미널에서 `code .` 실행 (가장 추천):**
 - Ubuntu 등 사용 중인 WSL 터미널을 실행합니다.
 - 작업할 프로젝트 폴더로 이동합니다. (예: `cd ~/projects/my-app`)
 - 해당 폴더에서 다음 명령어를 입력합니다.

```
code .
```

이 명령은 WSL 2 환경에 VS Code 서버를 자동으로 설치하고, Windows의 VS Code 클라이언트를 실행하여 해당 폴더를 열어줍니다.

- **방법 2: VS Code 원격 탐색기 사용:**
 - VS Code를 실행합니다.
 - 왼쪽 하단의 녹색 원격 연결 버튼(`<>`)을 클릭합니다.
 - 드롭다운 메뉴에서 "Connect to WSL" 또는 "Connect to WSL using Distro..."를 선택합니다.
 - 연결할 배포판(예: Ubuntu-22.04)을 선택하면 새로운 VS Code 창이 WSL에 연결된 상태로 열립니다.

연결이 성공하면 VS Code 왼쪽 하단의 녹색 버튼에 `WSL: <배포판이름>` 이 표시됩니다.

3단계: 개발 워크플로우 체험하기

이제 당신의 VS Code는 WSL 2와 완벽하게 연동되었습니다.

- **파일 탐색 및 편집:** VS Code의 탐색기(Explorer)를 통해 WSL의 파일 시스템에 있는 파일을 직접 열고 수정할 수 있습니다. 모든 변경 사항은 즉시 Linux 파일 시스템에 저장됩니다.
- **통합 터미널:** VS Code에서 새 터미널(단축키: `Ctrl+```)을 열면, Windows의 PowerShell이나 CMD가 아닌 WSL의 Bash 또는 Zsh 셸이 기본으로 열립니다. 여기서 `git`, `npm`, `python`` 등 Linux 명령어를 자유롭게 사용할 수 있습니다.
- **디버깅:** Node.js, Python, C++ 등 어떤 언어로 개발하든, VS Code의 강력한 디버거를 사용하여 WSL 환경에서 실행되는 애플리케이션을 직접 디버깅할 수 있습니다. Breakpoint를 설정하고 변수를 조사하는 모든 과정이 매끄럽게 동작합니다.
- **확장 프로그램 관리:** WSL에 연결된 상태에서 설치하는 확장 프로그램은 Windows가 아닌 WSL 환경 내에 설치됩니다. 이를 통해 Python, Docker, C/C++ 등 개발 환경에 필요한 확장 프로그램을 Windows와 분리하여 관리할 수 있습니다.

이제 여러분은 Windows의 쾌적한 UI와 Linux의 강력한 개발 생태계를 동시에 활용하는 최상의 개발 환경을 구축했습니다.